



同濟大學
TONGJI UNIVERSITY

计算机系统结构课程实验 总结报告

实验题目：简单的流水线 CPU 设计与性能分析

学号：2351520

姓名：毛星博

指导教师：陆有军

日期：2025 年 11 月 10 日

一、实验环境部署与硬件配置说明

操作系统环境：Windows 11 24H2 64 位

开发环境：vivado2016.2 + Modelsim

硬件配置：Xilinx NEXYS4 DDR Artix-7

二、实验的总体结构

1、5 级流水线结构 (IF, ID, EX, MEM, WB)

本次我设计的流水线 CPU 采用了经典的 5 级流水线 (Pipeline) 结构，将一条指令的执行过程分解为 **IF 取指**, **ID 译码**, **EX 执行**, **MEM 访存**, **WB 写回** 五个相对独立的阶段，每个阶段由一个时钟周期完成，从而允许多条指令重叠执行，提高 CPU 的吞吐率。

CPU 采用了 **哈佛结构**，将指令存储器 (IMEM) 和数据存储器 (DMEM) 分离。如代码 PCPU.v 所示，imem 模块在 IF 阶段被访问，dmem 模块在 MEM 阶段被访问，这允许 CPU 在同一周期内同时进行取指和数据读/写操作，避免了冯·诺依曼结构中的访存瓶颈。

五个阶段的功能说明如下：

- **IF (取指)：**根据程序计数器(PC)的值，从指令存储器(IMEM)中读取指令

主要逻辑：PC 寄存器在每个时钟周期更新 ($PC \leq PC_next$)。PC_next 默认指向 PC + 4，但在发生分支或跳转时会根据控制信号更新为目标地址。

- **ID (译码)：**对 IF 阶段传入的指令进行译码，生成后续阶段所需的控制信号。同时，从寄存器文件 (Regfile) 中读取源寄存器 (rs, rt) 的数据

主要逻辑：指令字段 (opcode, rs, rt, rd, imm 等) 被分离。控制信号 (ID_reg_write, ID_mem_read, ID_alu_op 等) 在此阶段生成。冒险检测单元 (Hazard Unit) 的主要判断逻辑也在此阶段进行。

- **EX (执行)：**执行算术或逻辑运算

主要逻辑：ALU (算术逻辑单元) 在此阶段工作。根据 ID_alu_src 信号选择 ALU 的第二个操作数 (来自寄存器或立即数)。数据转发逻辑 (forward_a, forward_b) 在此阶段生效，确保 ALU 获取最新的操作数值。

- **MEM (Memory Access - 访存)：**访问数据存储器 (DMEM)，进行读 (LOAD 指令) 或写 (STORE 指令) 操作

主要逻辑：dmem 模块在此阶段被使能 (ena)。若是 LOAD 指令，re=1，数据被读出；

若是 STORE 指令, we=1, EX_MEM_rt_data (来自寄存器的数据) 被写入内存。

- **WB (Write Back - 写回): 将执行结果或从内存中读取的数据写回到寄存器文件 (Regfile)**

主要逻辑: MEM_WB_reg_write 信号使能 regfile 的写操作。WB_write_data 选择器决定写回的数据来源: 对于 LOAD 指令, 来源是 MEM_WB_mem_data (内存数据); 对于算术/逻辑指令, 来源是 MEM_WB_alu_result (ALU 结果)。

为了在五个阶段之间传递数据和控制信号, 设计了**四个流水线寄存器**:

- **IF/ID 寄存器:**

IF_ID_PC: 存储当前指令的 PC 值, 用于计算分支跳转地址。

IF_ID_instr: 存储从 IMEM 取出的指令原文。

- **ID/EX 寄存器:**

ID_EX_PC: 传递 PC 值。

ID_EX_rs_data, ID_EX_rt_data: 存储从 Regfile 读出的原始数据。

ID_EX_imm_ext: 存储符号扩展后的 16 位立即数。

ID_EX_rs, ID_EX_rt, ID_EX_rd: 存储寄存器编号, 用于转发逻辑和 WB 阶段的写回。

控制信号: ID_EX_reg_write, ID_EX_mem_read, ID_EX_mem_write, ID_EX_alu_src, ID_EX_reg_dst, ID_EX_alu_op。

- **EX/MEM 寄存器:**

EX_MEM_alu_result: 存储 ALU 的运算结果, 用于访存地址或最终写回。

EX_MEM_rt_data: 存储需要写入内存的数据 (用于 STORE)。

EX_MEM_write_reg: 存储目标寄存器编号。

EX_MEM_zero, EX_MEM_negative: 存储 ALU 标志位

控制信号: EX_MEM_reg_write, EX_MEM_mem_read, EX_MEM_mem_write。

- **MEM/WB 寄存器:**

MEM_WB_alu_result: 传递 ALU 结果。

MEM_WB_mem_data: 存储从 DMEM 读出的数据 (用于 LOAD)。

MEM_WB_write_reg: 存储目标寄存器编号。

控制信号: MEM_WB_reg_write, MEM_WB_mem_read。

流水线执行会引入数据冒险和控制冒险。因此, 我通过设计**冒险检测单元 (Hazard_Unit)**

实现了转发、暂停和清空机制来解决这些问题。主要解决了数据冒险（RAW 写后读冲突），通过数据转发或暂停来实现；和控制毛线（当分支（BZ, BN）或跳转（JMP）指令在 ID 阶段才能确定是否跳转时，IF 阶段已经取了错误路径的指令），通过分支跳转判断和清空逻辑实现。具体的设计在第三部分架构部件的说明来详细阐述。

2、17 条指令的选取，指令格式的设计

本次设计的 CPU 支持一个包含 17 条指令的自定义指令集，采用定长指令编码，所有指令均为 32 位，根据指令类型分为 R 型指令、I 型指令、J 型指令三种格式。

指令列表如下：

类别	指令	操作码	格式	功能描述
特殊	NOP	0x00	R	空操作
	HALT	0x01	R	停机
算术	ADD	0x02	R	$rd = rs + rt$ (有符号)
	SUB	0x03	R	$rd = rs - rt$ (有符号)
	ADDI	0x0D	I	$rt = rs + imm$ (立即数加法)
逻辑	AND	0x04	R	$rd = rs \& rt$
	OR	0x05	R	$rd = rs rt$
	XOR	0x06	R	$rd = rs \wedge rt$
移位	SLL	0x07	R	$rd = rt \ll shamt$ (逻辑左移)
	SRL	0x08	R	$rd = rt \gg shamt$ (逻辑右移)
	SRA	0x09	R	$rd = rt \gg shamt$ (算术右移)
比较	CMP	0x0A	R	比较 rs 和 rt (ALU 设置标志位)
访存	LOAD	0x0B	I	$rd = MEM[rs + imm]$
	STORE	0x0C	I	$MEM[rs + imm] = rt$
分支	BZ	0x0E	I	if(Zero) $PC += imm \ll 2$ (零标志位为 1 则跳转)
	BN	0x0F	I	if(Negative) $PC += imm \ll 2$ (负标志位为 1 则跳转)
跳转	JMP	0x10	J	$PC = addr$ (无条件跳转)

指令格式设计如下：

(1) R 型指令：用于寄存器-寄存器操作 (ADD, SUB, AND, OR, SLL, CMP 等)

31	26 25	21 20	16 15	11 10	6 5	0
opcode	rs	rt	rd	shamt	保留	

opcode (6 位)：操作码，唯一标识指令

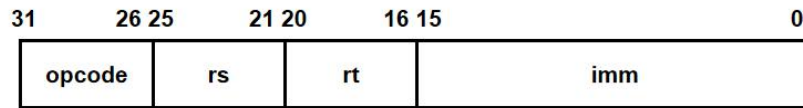
rs (5 位)：源寄存器 1 地址（0-31）

rt (5 位)：源寄存器 2 地址（0-31）

rd (5 位): 目标寄存器地址 (0-31)

shamt (5 位): 移位量 (0-31), 仅移位指令使用

(2) I 型指令: 用于带立即数的操作 (ADDI, LOAD, STORE, BZ, BN)



opcode (6 位): 操作码

rs (5 位): 源寄存器/基址寄存器

rt (5 位): 目标寄存器 (LOAD/ADDI) 或源寄存器 (STORE/分支)

imm (16 位): 立即数/偏移量/分支目标

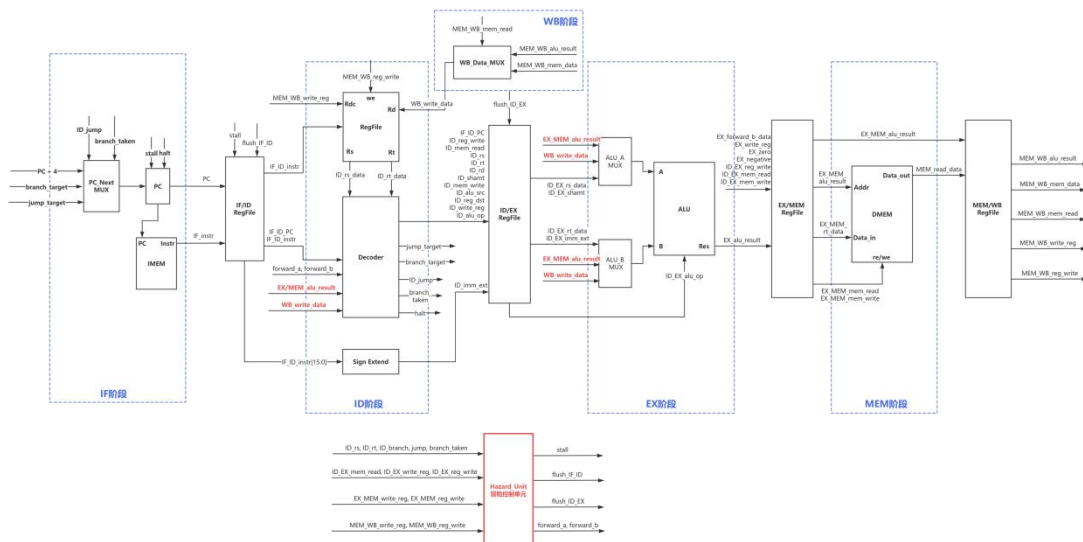
(3) J 型指令: 用于无条件跳转 (JMP)



opcode (6 位): 操作码

addr (26 位): 跳转目标地址 (字地址)

3、流水线 CPU 的总体架构图



三、 总体架构部件的解释说明

1、 IF 阶段

PC 是 32 位程序计数器，PC_plus_4 = PC + 4 计算顺序地址；PC_next 根据优先级依次选择 ID_jump(无条件跳转)、branch_taken(条件跳转)或顺序 PC_plus_4(查看 assign PC_next)。PC 只在 !stall && !halt 时更新，复位时被强制置为 0x00400000，确保指令从正确地址开始。通过 stall 信号避免冒险时 PC 前进，halt 控制停机后保持地址不变。

```
// ===== IF 阶段逻辑 =====
assign PC_plus_4 = PC + 4;

// PC 选择逻辑: 跳转 > 分支 > PC+4
assign PC_next = ID_jump ? jump_target :
                 branch_taken ? branch_target :
                 PC_plus_4;

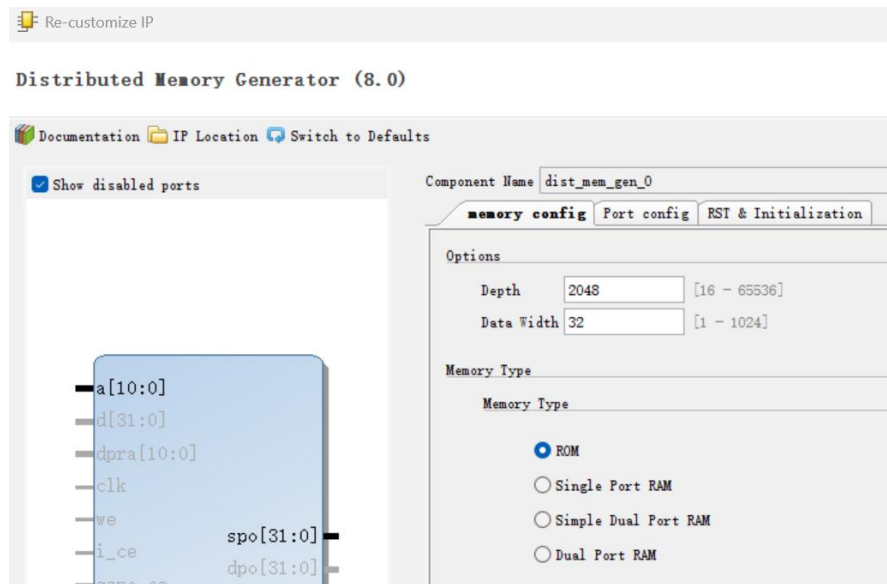
// PC 更新
always @(posedge clk or posedge rst) begin
    if (rst) begin
        PC <= 32'h00400000; // 初始 PC 地址
    end else if (!stall && !halt) begin
        PC <= PC_next;
    end
end
```

(1) IMEM

IMEM 接收当前 PC 值，内部计算 PC_addr。使用 Vivado 的 IP 核提供指令读取：输入地址 a，输出 Instr。

```
module IMEM(
    input [31:0] PC,
    output [31:0] Instr
);

wire [10:0] PC_addr; // 11 位指令码地址
assign PC_addr = PC[12:2] - 32'h00400000; // 计算 PC 地址,有一个映射关系
// 实例化 IP 核，输入指令码地址返回对应的指令
dist_mem_gen_0 Instr_mem(
    .a(PC_addr),
    .spo(Instr)
);
endmodule
```



(2) IF_ID 寄存器

IF_ID_PC 和 IF_ID_instr 分别保存当前周期取指的 PC 值和指令，供 ID 阶段解码用。

```
// IF/ID 流水线寄存器更新
always @(posedge clk or posedge rst) begin
    if (rst || flush_IF_ID) begin
        IF_ID_PC <= 32'h00400000;
        IF_ID_instr <= 32'h00000000; // NOP
    end else if (!stall) begin
        IF_ID_PC <= PC;
        IF_ID_instr <= IF_instr;
    end
end
end
```

2、ID 阶段

ID 阶段接收从 IF/ID 流水线寄存器传来的 PC 和指令，然后将指令拆解成 opcode、rs、rt、rd、imm、shamt 等字段。

(1) RegFile

RegFile 接收从 IF/ID 提取出的 rs/rt 地址，异步地输出对应的 Rs、Rt 数据供下一步操作。

写回阶段则通过 clk 上升沿处理 Rd 写入，写使能由 ID 期或 WB 期控制。

```
module RegFile(
    input clk, // 时钟信号
    input rst, // 复位信号
    input ena, // 使能信号
    input we, // 写使能信号
    input [4:0] Rdc, // 目标寄存器
    input [4:0] Rsc, // 源寄存器 1
    input [4:0] Rtc, // 源寄存器 2
```

```

input [31:0] Rd, // 数据输入
output [31:0] Rs, // 源寄存器 1 数据输出
output [31:0] Rt // 源寄存器 2 数据输出
);

```

(2) Decoder

三类控制信号（ID_reg_write、ID_mem_read、ID_mem_write、ID_alu_src、ID_reg_dst、ID_branch、ID_jump 等）都是根据 opcode 判断得来，比如算术/逻辑指令开启寄存器写、LOAD 使能 mem_read、STORE 使能 mem_write，立即数或访存指令选择立即数作为 ALU 第二操作数。Decoder 同时负责决定写回目标（rd 还是 rt）、是否触发分支/跳转，向 Hazard_Unit 提供 ID_branch/ID_rt/ID_rs 等信息以判断冒险。

```

// ===== ID 阶段逻辑 =====
// 指令译码
assign ID_opcode = IF_ID_instr[31:26];
assign ID_rs = IF_ID_instr[25:21];
assign ID_rt = IF_ID_instr[20:16];
assign ID_rd = IF_ID_instr[15:11];
assign ID_shamt = IF_ID_instr[10:6];
assign ID_imm = IF_ID_instr[15:0];
assign ID_addr = IF_ID_instr[25:0];

// 立即数符号扩展
assign ID_imm_ext = {{16{ID_imm[15]}}, ID_imm};

// 控制信号生成
// 写寄存器选择
// ALU 操作码生成
// 分支判断

```

(3) ID_EX 寄存器

ID/EX 保存 ID 阶段输出的数据（rs、rt、符号扩展后的立即数）、地址（rs/rt/rd）、shamt、opcode 以及一组控制信号（reg_write、mem_read、mem_write、alu_src、alu_op、write_reg）。

3、EX 阶段

(1) ALU

ALU 作为 EX 阶段的核心计算模块，被连接到两个 32 位输入 a/b（来自转发选择后的操作数）和 4 位控制码 aluc（由 ID 解码得到）。ALU 的输出既用于传给 EX/MEM（继续访存或写回），也通过 forward 路径回送给需要的后续指令。

```

module ALU(
    input [31:0] a, //32 位输入，操作数 1
    input [31:0] b, //32 位输入，操作数 2

```

```

input [3:0] aluc, //4 位输入, 控制 alu 的操作
output reg [31:0] r, //32 位输出, 由 a、b 经过 aluc 指定的操作生成
output reg zero, //0 标志位 (ZF)
output reg carry, // 进位标志位 (CF)
output reg negative, // 负数标志位 (SF)
output reg overflow // 溢出标志位 (OF)
);

```

(2) EX_MEM 寄存器

ALU 结果连同 forward 过来的 rt 数据写入 EX_MEM_reg。同时控制信号通过 ID_EX 传递给 EX/MEM。

```

// EX/MEM 流水线寄存器更新
always @(posedge clk or posedge rst) begin
    if (rst) begin
        EX_MEM_alu_result <= 32'h00000000;
        EX_MEM_rt_data <= 32'h00000000;
        EX_MEM_write_reg <= 5'b00000;
        EX_MEM_zero <= 1'b0;
        EX_MEM_negative <= 1'b0;
        EX_MEM_reg_write <= 1'b0;
        EX_MEM_mem_read <= 1'b0;
        EX_MEM_mem_write <= 1'b0;
    end else begin
        EX_MEM_alu_result <= EX_alu_result;
        EX_MEM_rt_data <= EX_forward_b_data;
        EX_MEM_write_reg <= EX_write_reg;
        EX_MEM_zero <= EX_zero;
        EX_MEM_negative <= EX_negative;
        EX_MEM_reg_write <= ID_EX_reg_write;
        EX_MEM_mem_read <= ID_EX_mem_read;
        EX_MEM_mem_write <= ID_EX_mem_write;
    end
end
end

```

4、MEM 阶段

(1) DMEM

```

module DMEM(
    input clk, // 时钟信号
    input ena, // 使能信号
    input we, // 写使能信号
    input re, // 读使能信号
    input [4:0] addr, // 数据地址 (字地址, 0-31)
    input [31:0] data_in, // 数据输入

```

```
output [31:0] data_out // 数据输出
);
```

(2) MEM_WB 寄存器

MEM_WB 保存 MEM 阶段的两个候选写回数据（一是 ALU 结果 EX_MEM_alu_result，一是 DMEM 读出的 MEM_read_data）、目标寄存器编号和控制信号（reg_write、mem_read）供 WB 阶段使用。

```
// MEM/WB 流水线寄存器更新
always @(posedge clk or posedge rst) begin
    if (rst) begin
        MEM_WB_alu_result <= 32'h00000000;
        MEM_WB_mem_data <= 32'h00000000;
        MEM_WB_write_reg <= 5'b00000;
        MEM_WB_reg_write <= 1'b0;
        MEM_WB_mem_read <= 1'b0;
    end else begin
        MEM_WB_alu_result <= EX_MEM_alu_result;
        MEM_WB_mem_data <= MEM_read_data;
        MEM_WB_write_reg <= EX_MEM_write_reg;
        MEM_WB_reg_write <= EX_MEM_reg_write;
        MEM_WB_mem_read <= EX_MEM_mem_read;
    end
end
```

5、WB 阶段

最终选择写回寄存器文件的值。MEM/WB 寄存器在前一阶段已经保存 ALU 结果、从 DMEM 读取的数据以及目标寄存器号与控制信号。WB 通过一个多路选择器（WB_write_data = MEM_WB_mem_read ? MEM_WB_mem_data : MEM_WB_alu_result）判断当前指令是否为 Load，如果是则写回内存数据，否则写回 ALU 运算结果。

```
// ===== WB 阶段逻辑 =====
// 写回数据选择：内存数据或 ALU 结果
assign WB_write_data = MEM_WB_mem_read ? MEM_WB_mem_data : MEM_WB_alu_result;
```

6、冒险冲突检测单元 Hazard_Unit

输入：当前 ID 阶段的源寄存器 ID_rs/ID_rt、分支标志、ID/EX/EX/MEM/MEM/WB 各阶段的写寄存器号与写使能、以及控制信号（branch_taken、jump）。

输出：

stall：由 Load-Use 或分支冒险触发，暂停 IF/ID/PC 前进；

flush_IF_ID：分支跳转或 jump 时清空 IF/ID；

flush_ID_EX: 跳转时清空; 在 stall 时插入气泡 (控制信号清零但保留数据);

branch_taken 在 stall 为 0 的前提下才计算, 防止在冒险期间误跳转。

这个单元保证了流水线在面对 RAW、Load-Use、控制冒险时要么通过转发直接提供数据, 要么通过 stall/flush 重新安排指令序列, 维持正确性。

```
module Hazard_Unit(  
    // ID 阶段寄存器  
    input [4:0] ID_rs,           // ID 阶段源寄存器 1  
    input [4:0] ID_rt,           // ID 阶段源寄存器 2  
    input ID_branch,             // ID 阶段是否为分支指令 (BZ/BN)  
  
    // ID/EX 阶段信号  
    input [4:0] ID_EX_rt,  
    input ID_EX_mem_read,        // ID/EX 阶段内存读信号  
    input [4:0] ID_EX_write_reg, // ID/EX 阶段写寄存器  
    input ID_EX_reg_write,        // ID/EX 阶段寄存器写使能  
  
    // EX/MEM 阶段信号  
    input [4:0] EX_MEM_write_reg, // EX/MEM 阶段写寄存器  
    input EX_MEM_reg_write,        // EX/MEM 阶段寄存器写使能  
  
    // MEM/WB 阶段信号  
    input [4:0] MEM_WB_write_reg, // MEM/WB 阶段写寄存器  
    input MEM_WB_reg_write,        // MEM/WB 阶段寄存器写使能  
  
    // 控制冒险信号  
    input branch_taken,           // 分支跳转信号  
    input jump,                   // 无条件跳转信号  
  
    // 输出控制信号  
    output reg stall,             // 流水线暂停信号  
    output reg [1:0] forward_a,   // 转发控制 A (00:无转发, 01:从 WB 转发, 10:从 MEM 转发)  
    output reg [1:0] forward_b,   // 转发控制 B (00:无转发, 01:从 WB 转发, 10:从 MEM 转发)  
    output reg flush_IF_ID,        // 清空 IF/ID 流水线寄存器  
    output reg flush_ID_EX         // 清空 ID/EX 流水线寄存器  
);
```

四、实验仿真过程

1、指令流水线的仿真过程

实验仿真时, 选择后续性能测试板块的比萨塔摔鸡蛋测试来作为测试用例。首先为比萨塔摔鸡蛋测试编写 c 语言程序作为基准, 该程序在标准 PC 环境下编译运行, 得到一组基准

结果。然后对照 C 语言逻辑和自定义的 17 条指令集，手动将 C 程序翻译为汇编语言文件。由于 CPU 采用了自定义的指令集架构，MARS 汇编器无法使用。为此，我编写了一个 Python 汇编器来将 asm 转化为 coe。（以下是部分代码）

```
def encode_r_type(opcode, rd, rs, rt, shamt=0):
    return (opcode << 26) | (rs << 21) | (rt << 16) | (rd << 11) | (shamt << 6)

def encode_i_type(opcode, rs, rt, immediate):
    return (opcode << 26) | (rs << 21) | (rt << 16) | (immediate & 0xFFFF)

def encode_j_type(opcode, address):
    return (opcode << 26) | (address & 0x3FFFFFFF)
```

在 Vivado 中，将 IMEM 模块的 IP 核配置为从该 coe 文件加载初始 ROM 内容。

2、编写 testbench

为了自动化验证“摔鸡蛋”程序的执行结果，我编写了 PCPU_tb.v 测试平台。在 CPU 停机且流水线排空后，Testbench 直接访问 CPU 内部 DMEM 模块的存储阵列 (uut.dmem.mem[i])。testbench 会打印出程序使用的关键内存地址及其内容。

```
// 摔鸡蛋测试 - Drop Eggs Test
module PCPU_tb_drop_eggs();
    reg clk;
    reg rst;
    wire halt;
    integer i;
    integer cycle_count;

    // 实例化 CPU
    PCPU uut (
        .clk(clk),
        .rst(rst),
        .halt(halt)
    );

    // 时钟生成 - 10ns 周期
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // 周期计数器
    initial begin
        cycle_count = 0;
```

```

    forever begin
        @(posedge clk);
        if (!rst && !halt) cycle_count = cycle_count + 1;
    end
end

// 测试流程
initial begin
    $display("=====");
    $display("  PCPU Egg Drop Problem Test");
    $display("=====");
    $display("Parameters: 100 floors, secret threshold = 37");
    $display("=====\\n");

    // 复位
    rst = 1;
    #20;
    rst = 0;
    $display("[%0t ns] CPU started", $time);

    // 等待 halt 信号
    wait(halt == 1);
    $display("[%0t ns] CPU halted after %0d cycles", $time, cycle_count);

    // 等待流水线完成
    repeat(10) @(posedge clk);

    // 输出结果
    $display("\\n=====");
    $display("  Register File Contents (Results)");
    $display("=====");
    $display("Reg  | Decimal Value | Hex Value  | Description");
    $display("-----|-----|-----|-----");

    // Output register results
    $display("$s0  | %13d | 0x%08h | SECRET_THRESHOLD", uut.regfile.array_reg[16],
uut.regfile.array_reg[16]);
    $display("$s1  | %13d | 0x%08h | TOTAL_FLOORS", uut.regfile.array_reg[17],
uut.regfile.array_reg[17]);
    $display("$s2  | %13d | 0x%08h | low", uut.regfile.array_reg[18],
uut.regfile.array_reg[18]);
    $display("$s3  | %13d | 0x%08h | [OUT] Threshold found",
uut.regfile.array_reg[19], uut.regfile.array_reg[19]);

```

```

        $display("$s4 | %13d | 0x%08h | current_floor", uut.regfile.array_reg[20],
uut.regfile.array_reg[20]);
        $display("$s5 | %13d | 0x%08h | [OUT] m_up", uut.regfile.array_reg[21],
uut.regfile.array_reg[21]);
        $display("$s6 | %13d | 0x%08h | [OUT] n_down", uut.regfile.array_reg[22],
uut.regfile.array_reg[22]);
        $display("$s7 | %13d | 0x%08h | [OUT] h_broken", uut.regfile.array_reg[23],
uut.regfile.array_reg[23]);
        $display("$t8 | %13d | 0x%08h | [OUT] last_drop_broken",
uut.regfile.array_reg[24], uut.regfile.array_reg[24]);
        $display("$t9 | %13d | 0x%08h | [OUT] Cost1 (f1)", uut.regfile.array_reg[25],
uut.regfile.array_reg[25]);
        $display("$t0 | %13d | 0x%08h | [OUT] Cost2 (f2)", uut.regfile.array_reg[8],
uut.regfile.array_reg[8]);

        $display("=====\n");

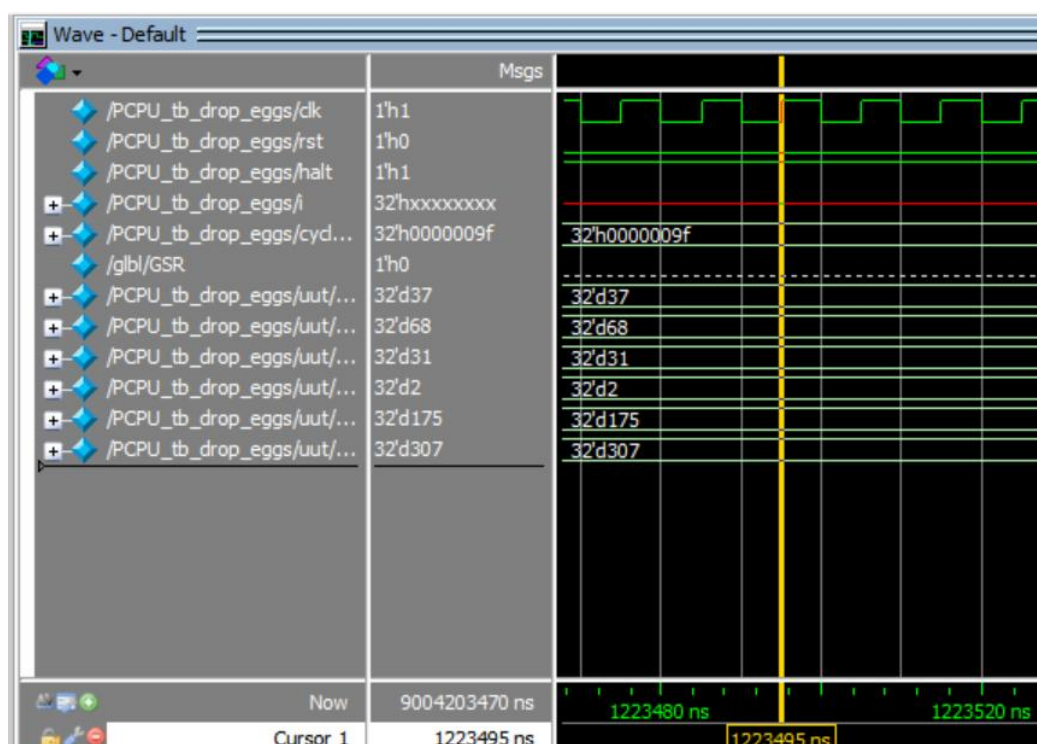
    $finish;
end

endmodule

```

五、实验仿真的波形图及某时刻寄存器值的物理意义

1、实验仿真的波形图和控制台输出



Register File Contents (Results)			
Reg	Decimal Value	Hex Value	Description
\$s0	37	0x00000025	SECRET_THRESHOLD
\$s1	100	0x00000064	TOTAL_FLOORS
\$s2	38	0x00000026	low
\$s3	37	0x00000025	[OUT] Threshold found
\$s4	37	0x00000025	current_floor
\$s5	68	0x00000044	[OUT] m_up
\$s6	31	0x0000001f	[OUT] n_down
\$s7	2	0x00000002	[OUT] h_broken
\$t8	0	0x00000000	[OUT] last_drop_broken
\$t9	175	0x000000af	[OUT] Cost1 (f1)
\$t0	307	0x00000133	[OUT] Cost2 (f2)

Testbench 在 CPU 停机后，通过 \$display 语句打印出的 RegFile 内容，和波形图显示的一致：

\$s5 (m_up) 的十进制值为 68。

\$s6 (n_down) 的十进制值为 31。

\$s7 (h_broken) 的十进制值为 2。

\$t9 (Cost1) 的十进制值为 175。

\$t0 (Cost2) 的十进制值为 307。

2、某时刻寄存器值的物理意义

Edit		Execute		Registers										
				Coproc 1	Coproc 0									
Test Segment														
Bpt	Address	Code	Basic	Source										
	0x04000000	0x04000004	add \$t0,\$0,0x00000004	28	li \$t0, 4									
	0x04000004	0x04000008	lwi \$t1,0x00000000	29	li \$t1, 0x00000000									
	0x04000008	0x0400000c	wri \$t1,\$0	31	swi \$t1,\$0									
	0x0400000c	0x00000000	rsyncall	30	rsyncall									
	0x0400000d	0x04000005	add \$t0,\$0,0x00000005	33	li \$t0, 37	# \$t0 = 37 (SEPRST_DUMP_SIZE)								
	0x04000014	0x0400001a	addi \$t1,\$0,0x0000000a	34	li \$t1, 100	# \$t1 = 100 (TOTAL_FLOORS)								
	0x04000012	0x04000005	add \$t0,\$0,\$t1	35	li \$t0, 0	# \$t0 = 0 (last)								
	0x04000010	0x00110021	addi \$t0,\$0,\$t1	36	swi \$t0,\$t1	# \$t0 = 100 (high TOTAL_FLOORS)								
	0x04000012	0x04000005	add \$t0,\$0,\$t1	37	li \$t0, 0	# \$t0 = 0 (current floor)								
	0x04000004	0x04000005	add \$t0,\$0,\$t1	38	li \$t0, 0	# \$t0 = 0 (n_g)								
	0x04000005	0x04000005	add \$t0,\$0,\$t1	39	li \$t0, 0	# \$t0 = 0 (n_down)								
	0x04000004	0x04000005	add \$t0,\$0,\$t1	40	li \$t0, 0	# \$t0 = 0 (is_broken)								
	0x04000005	0x04000005	add \$t0,\$0,\$t1	41	li \$t0, -1	# \$t0 = -1 (last_dry_broken)								
	0x04000034	0x0724021a	addi \$t0,\$0,\$t1	46	add \$t0,\$t0,\$t2									
	0x04000030	0x17200016	hsw \$t0,\$t0	48	hsw \$t0,\$t0									
	0x04000032	0x18400001	hsw \$t0,\$t0	54	hsw \$t0,\$t0									
Data Segment														
Address				Value (+0)	Value (+1)	Value (+2)	Value (+10)	Value (+1)	Value (+1)	Value (+1)	Value (+1)			
	0x10010000	0x73911433	0x08542074	0x02039473	0x01146f54	0x06460206	0x7372666f	0x00302020	0x03030200	0x03030200	20			
	0x10010002	0x08572636	0x08574207	0x06778872	0x02444646	0x04000000	0x04000000	0x04000000	0x04000000	0x04000000	21			
	0x10010004	0x04085610	0x08573852	0x02046646	0x04046646	0x04046646	0x02030464	0x04174694	0x04174694	0x04174694	24			
	0x10010006	0x08770638	0x4d032428	0x0206576f	0x02039473	0x04002000	0x0206f764	0x04174694	0x04000020	0x04000020	26			
	0x10010008	0x08774202	0x02046646	0x02046646	0x02040234	0x04000000	0x04174694	0x04174694	0x04174694	0x04174694	28			
	0x1001000a	0x46167512	0x00010204	0x04047242	0x03030645	0x06097275	0x00046516	0x04000044	0x04174643	0x04174643	29			
	0x10010010	0x02030210	0x06f73032	0x08583723	0x08583723	0x04046516	0x06097275	0x02020646	0x02030240	0x02030240	27			
	0x1001000e	0x02030200	0x02032207	0x04000000	0x04000000	0x04000000	0x04000000	0x04000000	0x04000000	0x04000000	28			
	0x10010010	0x02030200	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	31			
	0x10010012	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	32			
	0x10010014	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	33			
	0x10010016	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	34			
	0x10010018	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	35			
	0x1001001a	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	36			
	0x1001001c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	37			
	0x1001001e	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	38			
	0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	39			
	0x10010022	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	40			
	0x10010024	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	41			
	0x10010026	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	42			
	0x10010028	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	43			
	0x1001002a	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	44			
	0x1001002c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	45			
	0x1001002e	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	46			
	0x10010030	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	47			
	0x10010032	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	48			
	0x10010034	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	49			
	0x10010036	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	50			
	0x10010038	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	51			
	0x1001003a	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	52			
	0x1001003c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	53			
	0x1001003e	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	54			
	0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	55			
	0x10010042	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	56			
	0x10010044	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	57			
	0x10010046	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	58			
	0x10010048	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	59			
	0x1001004a	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	60			
	0x1001004c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	61			
	0x1001004e	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	62			
	0x10010050	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	63			
	0x10010052	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	64			
	0x10010054	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	65			
	0x10010056	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	66			
	0x10010058	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	67			
	0x1001005a	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	68			
	0x1001005c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	69			
	0x1001005e	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	70			
	0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	71			
	0x10010062	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	72			
	0x10010064	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	73			
	0x10010066	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	74			
	0x10010068	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	75			
	0x1001006a	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	76			
	0x1001006c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	77			
	0x1001006e	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	78			
	0x10010070	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	79			
	0x10010072	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	80			
	0x10010074	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	81			
	0x10010076	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	82			
	0x10010078	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	83			
	0x1001007a	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	84			
	0x1001007c	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	85			
	0x1001007e	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	86			
	0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	87			

为了验证本 CPU 寄存器值的正确性，使用 C 语言基准程序、MARS MIPS 模拟器进行交叉对比。下表对上述三个来源（本 CPU、MARS、C 语言）的最终结果进行了详细比对：

寄存器	物理意义	CPU	MARS	C 语言	结果
\$s0 (16)	SECRET_THRESHOLD	37 (0x25)	37 (0x25)	37	Match
\$s1 (17)	TOTAL_FLOORS	100 (0x64)	100 (0x64)	100	Match
\$s2 (18)	low (循环变量)	38 (0x26)	38 (0x26)	38	Match
\$s3 (19)	[OUT] Threshold found	37 (0x25)	37 (0x25)	37	Match
\$s4 (20)	current_floor	37 (0x25)	37 (0x25)	37	Match
\$s5 (21)	[OUT] m_up	68 (0x44)	68 (0x44)	68	Match
\$s6 (22)	[OUT] n_down	31 (0x1f)	31 (0x1f)	31	Match
\$s7 (23)	[OUT] h_broken	2 (0x02)	2 (0x02)	2	Match
\$t8 (24)	[OUT] last_drop_broken	0 (0x00)	0 (0x00)	0 (没破)	Match
\$t9 (25)	[OUT] Cost1 (f1)	175 (0xaf)	175 (0xaf)	175	Match
\$t0 (8)	[OUT] Cost2 (f2)	307 (0x133)	307 (0x133)	307	Match

在 Vivado 中运行后，RegFile 的所有寄存器的最终值，与 MARS MIPS 模拟器运行相同汇编程序的结果完全相同。也与 C 语言基准程序的输出完全匹配。证明了本流水线 CPU 设计在功能上是完全正确的。

六、流水线 CPU 实验性能验证模型

实验性能验证模型：比萨塔摔鸡蛋游戏。两个同学在可变换层数的比萨塔上摔鸡蛋，一个同学秘密设定同一批鸡蛋耐摔值；另一个同学在指定层高的比萨塔拿着鸡蛋往下摔，用最少的摔次数和摔破的鸡蛋数求出鸡蛋的耐摔值。假定在耐摔值的楼层及其下面楼层，鸡蛋摔不破，可以重复使用，否则鸡蛋摔破。要求模型的算法输出包括：摔的总次数、摔的总鸡蛋数、最后摔的鸡蛋是否摔破。用你的模型评价该游戏在两个不同历史时期花费的总成本 $f=m*p1+n*p2+h*p3$ ， m 为上的楼层总数， n 为下的楼层总数， h 为摔破的鸡蛋总数， $p1$ 为每上 1 层的成本， $p2$ 为每下 1 层的成本， $p3$ 为每个鸡蛋的成本；在物质匮乏时期， $p1=2$ ， $p2=1$ ， $p3=4$ ；在人力成本增长时期， $p1=4$ ， $p2=1$ ， $p3=2$ 。请使用 C 语言设计该验证模型的算法，并把 C 语言汇编为 MIPS 或 RISC-V 指令汇编程序，同时利用编译器生成 MIPS 或 RISC-V 指令集可执行目标程序。

编写的 c 语言程序如下：

```
int main()
{
    int low = 0, high = TOTAL_FLOORS;
    int current_floor = 0;
    int m_up = 0, n_down = 0, h_broken = 0, drops = 0;
    int last_drop_broken = -1;
    printf("开始测试: 比萨塔层数: %d, 目标耐摔值: %d\n\n", TOTAL_FLOORS, SECRET_THRESHOLD);
    while (low <= high)
    {
        if (low == 0 && high == 0)
            break;
```

```

    int mid = (low + high + 1) / 2;
    drops++;
    // 计算移动成本
    if (mid > current_floor)
        m_up += (mid - current_floor);
    else
        n_down += (current_floor - mid);
    current_floor = mid;
    // --- 测试并记录最后状态 ---
    if (current_floor > SECRET_THRESHOLD) {
        h_broken++;
        high = mid - 1;
        last_drop_broken = 1; // 记录: 这次破了
        printf("第 %d 次摔鸡蛋: 摔在第 %d 层 -> 【破了】\n", drops, current_floor);
    } else {
        low = mid + 1;
        last_drop_broken = 0; // 记录: 这次没破
        printf("第 %d 次摔鸡蛋: 摔在第 %d 层 -> 【没破】\n", drops, current_floor);
    }
}

printf("测试结束。测得耐摔值: %d\n", high);
// 输出最后一次的状态
if (last_drop_broken == 1)
    printf("最后一次摔的鸡蛋: 【破了】\n");
else
    printf("最后一次摔的鸡蛋: 【没破】\n");
printf("统计 -> 上楼: %d, 下楼: %d, 破蛋总数: %d\n\n", m_up, n_down, h_broken);
int f1 = m_up * 2 + n_down * 1 + h_broken * 4;
int f2 = m_up * 4 + n_down * 1 + h_broken * 2;
printf("资源匮乏时期总成本      : %d\n", f1);
printf("人力资源增长时期总成本: %d\n", f2);
return 0;
}

```

程序采用二分查找算法，在每一次循环中，算法通过 $mid = (low + high + 1) / 2$ 找到中间楼层，然后根据测试结果（破了或没破），将搜索区间 $[low, high]$ 的大小缩小一半（通过 $high = mid - 1$ 或 $low = mid + 1$ ）。时间复杂度： $O(\log N)$ ；空间复杂度： $O(1)$ 。

对照 C 语言逻辑和自定义的 17 条指令集，手动将 C 程序翻译为汇编语言文件：

```

# 寄存器：
# $s0 (16): SECRET_THRESHOLD (秘密阈值)
# $s1 (17): TOTAL_FLOORS (总楼层)
# $s2 (18): low (最低楼层)
# $s3 (19): high (最高楼层)
# $s4 (20): current_floor (当前楼层)
# $s5 (21): m_up (向上移动次数)

```

```

# $s6 (22): n_down (向下移动次数)
# $s7 (23): h_broken (碎蛋数量)
# $t8 (24): last_drop_broken (上次是否碎蛋)
#
# 最终结果:
# $s3 (19): Threshold found (最终 threshold)
# $s5 (21): m_up (向上移动次数)
# $s6 (22): n_down (向下移动次数)
# $s7 (23): h_broken (碎蛋次数)
# $t8 (24): last_drop_broken (最后一次掉落结果)
# $t9 (25): Cost1 (f1 = m*2 + n*1 + h*4)
# $t0 (8): Cost2 (f2 = m*4 + n*1 + h*2)
# =====

# --- 初始化 ---
    addi $s0, $zero, 37    # $s0 = 37 (秘密阈值)
    addi $s1, $zero, 100   # $s1 = 100 (总楼层)
    addi $s2, $zero, 0     # $s2 = 0 (low)
    add  $s3, $s1, $zero    # $s3 = 100 (初始化 high)
    addi $s4, $zero, 0     # $s4 = 0 (当前楼层)
    addi $s5, $zero, 0     # $s5 = 0 (向上移动计数)
    addi $s6, $zero, 0     # $s6 = 0 (向下移动计数)
    addi $s7, $zero, 0     # $s7 = 0 (碎蛋计数)
    addi $t8, $zero, -1    # $t8 = -1 (初始为未知)

# --- 主循环: low <= high ---
LOOP_START:
    # 检查循环条件: 若 high < low 则结束
    # BN $s3, $s2, LOOP_END 表示若 $s3 < $s2 则跳转
    bn  $s3, $s2, LOOP_END

    # 特殊判断: low 和 high 同为 0 时退出
    bz  $s2, $zero, CHECK_HIGH_ZERO
    jmp NORMAL_LOOP

CHECK_HIGH_ZERO:
    bz  $s3, $zero, LOOP_END

NORMAL_LOOP:
    # 计算中点 mid = (low + high + 1) / 2
    add $t0, $s2, $s3      # $t0 = low + high
    addi $t0, $t0, 1        # $t0 = low + high + 1
    sra $t0, $t0, 1        # $t0 = mid (算术右移)

    # 计算移动距离

```

```

# 若当前楼层低于 mid, 跳转向上
bn    $s4, $t0, MOVE_UP

# 否则判断是否需要向下或保持不动
# 检查 current_floor 是否等于 mid
# 相等则保持, 不等则向下移动
sub    $t1, $s4, $t0    # $t1 = current_floor - mid
bz     $t1, $zero, UPDATE_FLOOR # diff 为 0 时不移动

# 向下移动: n_down 加上差值
# 此时 $t1 已经是正数
add    $s6, $s6, $t1    # n_down += $t1
jmp    UPDATE_FLOOR

MOVE_UP:
# 向上移动: m_up 增加 mid 与 current_floor 的差值
sub    $t1, $t0, $s4    # $t1 = mid - current_floor
add    $s5, $s5, $t1    # m_up += $t1

UPDATE_FLOOR:
add    $s4, $t0, $zero  # current_floor = mid

# --- 核心测试: 扔鸡蛋 ---
# 若当前楼层高于秘密阈值则蛋破
bn     $s0, $s4, EGG_BROKEN

EGG_SURVIVED:
# 蛋未破碎, 调整 low 和标志
addi   $s2, $t0, 1      # low = mid + 1
addi   $t8, $zero, 0    # last_drop_broken = 0
jmp    LOOP_START

EGG_BROKEN:
# 蛋破碎, 更新相关统计
addi   $s7, $s7, 1      # h_broken++
addi   $s3, $t0, -1     # high = mid - 1
addi   $t8, $zero, 1    # last_drop_broken = 1
jmp    LOOP_START

LOOP_END:
# --- 计算代价 (结果仍保留在寄存器) ---
# Cost1:  $f1 = m*2 + n*1 + h*4$ , 保存到 $t9
sll    $t1, $s5, 1      # $t1 = m * 2
sll    $t2, $s7, 2      # $t2 = h * 4

```

```

add $t9, $t1, $s6      # $t9 = (m*2) + n
add $t9, $t9, $t2      # $t9 = Cost1 ((m*2 + n) + (h*4))

# Cost2: f2 = m*4 + n*1 + h*2, 保存到 $t0
sll $t1, $s5, 2        # $t1 = m * 4
sll $t2, $s7, 1        # $t2 = h * 2
add $t0, $t1, $s6      # $t0 = (m*4) + n
add $t0, $t0, $t2      # $t0 = Cost2 ((m*4 + n) + (h*2))

# 最终结果分别保存在:
# $s3 = threshold found, $s5 = m_up, $s6 = n_down
# $s7 = h_broken, $t8 = last_drop_broken
# $t9 = Cost1, $t0 = Cost2

halt                  # 结束程序

```

使用自定义汇编器将汇编程序转换为机器码（部分截图）：

```

≡ drop_eggs.coe
1  memory_initialization_radix=16;
2  memory_initialization_vector=
3  34100025,
4  34110064,
5  34120000,
6  0a209800,
7  34140000,
8  34150000,
9  34160000,
10 34170000,
11 3418ffff,
12 3e720016,
13 3a400001,
14 4010000d,
15 3a600013,

```

七、实验验算程序下板测试过程与实现

1、设置 xdc 约束

复位信号 rst 连接到 N17，按下后复位；halt 停机信号连接到 H17，当 halt 信号高电平时指示灯亮。同时连接七段数码管的信号。

另外，需要为 CPU 添加时钟约束。设置 create clock、Input Delay 和 Output Delay。时序分析正确：

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 2.967 ns	Worst Hold Slack (WHS): 0.113 ns	Worst Pulse Width Slack (WPWS): 5.000 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 181	Total Number of Endpoints: 181	Total Number of Endpoints: 88	
All user specified timing constraints are met.			

```

# 时钟和复位信号
set_property PACKAGE_PIN E3 [get_ports clk]

```

```
set_property PACKAGE_PIN N17 [get_ports rst]
set_property PACKAGE_PIN H17 [get_ports halt]

# 七段数码管段选信号
set_property PACKAGE_PIN T10 [get_ports {seg[0]}]
set_property PACKAGE_PIN R10 [get_ports {seg[1]}]
set_property PACKAGE_PIN K16 [get_ports {seg[2]}]
set_property PACKAGE_PIN K13 [get_ports {seg[3]}]
set_property PACKAGE_PIN P15 [get_ports {seg[4]}]
set_property PACKAGE_PIN T11 [get_ports {seg[5]}]
set_property PACKAGE_PIN L18 [get_ports {seg[6]}]
set_property PACKAGE_PIN H15 [get_ports {seg[7]}]

# 七段数码管位选信号
set_property PACKAGE_PIN J17 [get_ports {an[0]}]
set_property PACKAGE_PIN J18 [get_ports {an[1]}]
set_property PACKAGE_PIN T9 [get_ports {an[2]}]
set_property PACKAGE_PIN J14 [get_ports {an[3]}]
set_property PACKAGE_PIN P14 [get_ports {an[4]}]
set_property PACKAGE_PIN T14 [get_ports {an[5]}]
set_property PACKAGE_PIN K2 [get_ports {an[6]}]
set_property PACKAGE_PIN U13 [get_ports {an[7]}]

# IO 标准设置 - 位选
set_property IOSTANDARD LVCMOS33 [get_ports {an[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {an[0]}]

# IO 标准设置 - 段选
set_property IOSTANDARD LVCMOS33 [get_ports {seg[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {seg[0]}]

# IO 标准设置 - 时钟和复位
```

```

set_property IOSTANDARD LVCMOS33 [get_ports rst]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports halt]

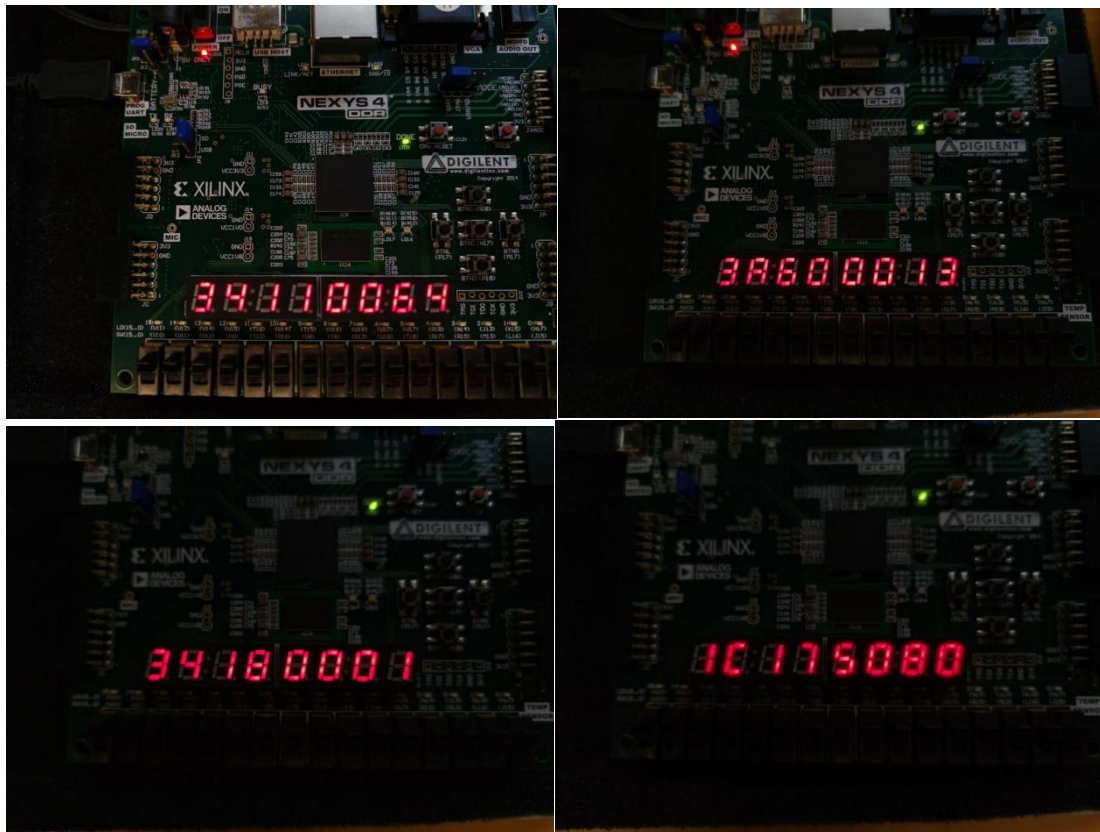
create_clock -period 11.000 -name clk_pin -waveform {0.000 5.500} [get_ports clk]
set_input_delay -clock [get_clocks clk_pin] 1.000 [get_ports rst]
set_output_delay -clock [get_clocks clk_pin] 0.000 [get_ports -filter { NAME =~ "*" && DIRECTION
== "OUT" }]

```

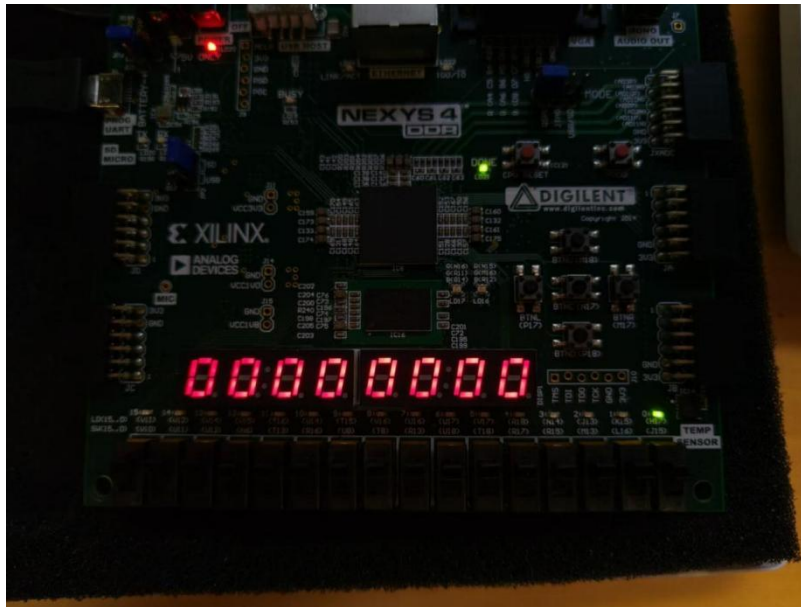
2、下板验证

按下复位信号后，数码管每隔一个时钟周期（用分频器将 CPU 时钟周期调整为 1 秒）更新一次，显示 IMEM 当前读取的指令码。

如图，刚开始执行的第 2 条指令显示为 34110064，与 coe 文件一致。34110064 转为二进制是 0011 0100 0001 0001 0000 0000 0110 0100，对应汇编指令 `addi $s1, $zero, 100`。3A600013 转为二进制是 0011 1010 0110 0000 0000 0000 0001 0011，对应汇编指令 `bz $s3, $zero, 19`。其他同理。



当指令执行完后，指令显示为全 0，Halt 指示灯（H17）亮。



八、流水线的性能指标定性分析（包括：吞吐率、加速比、效率及相关与冲突分析、CPU 的运行时间及存储器空间的使用）

1、吞吐率分析

```
[20000 ns] CPU started
[1605000 ns] CPU halted after 159 cycles
Dynamic instruction count: 109
Throughput (TP) = 109 / 159 = 0.686
CPI (Cycles Per Instruction) = 159 / 109 = 1.459
```

吞吐率是指单位时间内完成的指令数量。通过在 tb 文件中添加时钟周期和指令条数的计数器，可以得到执行的时钟周期数为 159，动态执行指令条数为 109。则吞吐率可计算：

$$TP = \frac{\text{动态执行指令条数}}{\text{时钟周期数}} = \frac{109}{159} = 0.685535$$

2、加速比分析

将 5 级流水线 CPU 与一个等效的多周期 CPU 进行比较。R-type 所需 4 周期；I-type (ALU)4 周期；I-type (Branch) 3 周期；J-type (Jump)3 周期。

计算总周期数：

R-type: 43 条 * 4 周期/条 = 172 周期

I-type (addi): 31 条 * 4 周期/条 = 124 周期

Branch: 23 条 * 3 周期/条 = 69 周期

Jump: 12 条 * 3 周期/条 = 36 周期

总周期数: $172 + 124 + 69 + 36 = 401$ 周期

加速比可计算:

$$S = \frac{\text{顺序执行所需时钟周期数}}{\text{流水线CPU时钟周期数}} = \frac{401}{159} = 2.52201$$

3、效率分析

流水线效率是衡量流水线各阶段被有效利用的程度。

$$E = \frac{\text{顺序执行所需时钟周期数}}{\text{流水线CPU时钟周期数} \times 5} = \frac{401}{159 \times 5} = 0.504403$$

4、相关与冲突分析

本流水线 CPU 设计通过硬件机制，处理了 2 种类型的冒险，但不同处理方式对性能的影响不同:

(1) 数据冒险

当一条指令（如 SUB）需要使用前一条指令（如 ADD）尚未写回的结果时发生(RAW)。采用数据转发实现了完善的转发逻辑（EX/MEM -> EX, MEM/WB -> EX）。当检测到 RAW 冒险时，ALU 的输入 MUX 会直接从流水线寄存器获取最新数据，而不是等待 WB 阶段。绝大多数数据冒险被完美隐藏，CPI_stall 惩罚为 0。

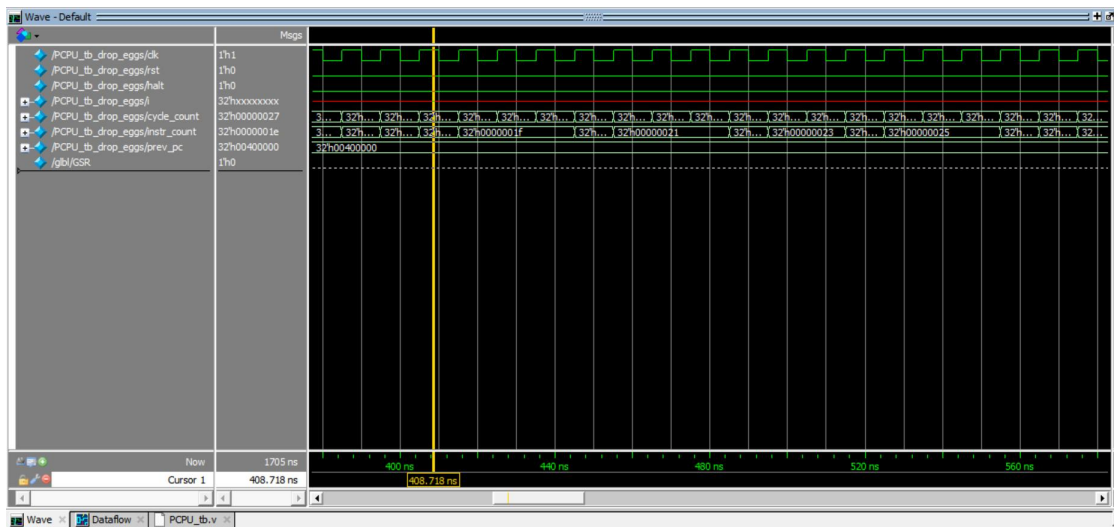
唯一的例外是 Load-Use 冒险。LOAD 指令在 MEM 阶段才能取回数据，无法转发给紧随其后在 EX 阶段就需要该数据的指令。Hazard_Unit 检测到此情况会强制流水线暂停 1 个周期 (插入一个气泡)。这是 CPI_stall 的主要来源之一。

(2) 控制冲突

BZ、BN、JMP 等指令在 ID 阶段才能确定是否跳转，但 IF 阶段已经取了 PC+4 的指令。采用“分支判断在 ID 级”和“默认预测不跳转”的策略。当分支（branch_taken）或跳转（jump）确实发生时，Hazard_Unit 会触发流水线清空 (Flush)，废弃 IF 和 ID 阶段的错误指令。每次跳转成功，都会导致 2 个周期的性能损失（IF 和 ID 阶段被清空）。“摔鸡蛋”程序中存在大量 while 循环（BZ）和 if 判断（BN），这些跳转导致的 Flush 是 CPI_stall 的另一个主要来源。

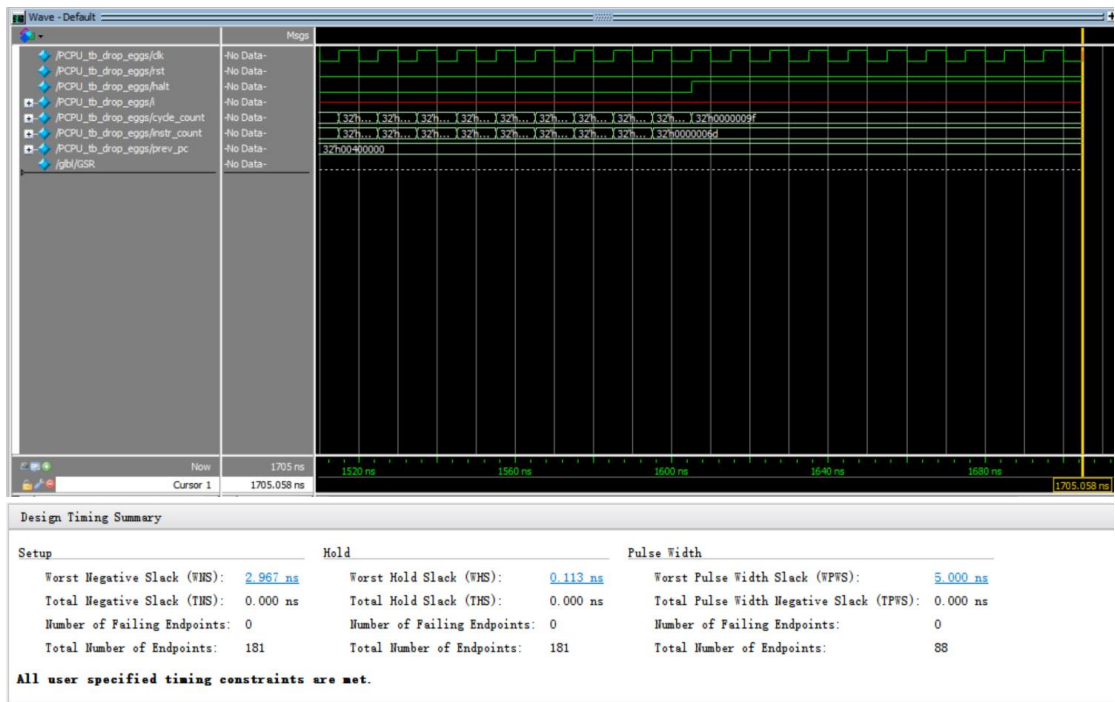
如下图所示，时钟周期数增加但指令数不变的情况（比如一条指令两个或三个周期），就是流水线暂停发生。Load-Use 冒险会使流水线暂停 1 个周期，分支（branch_taken）或

跳转（jump）发生会导致 2 个周期的性能损失。



5、CPU 的运行时间

时钟周期 10ns，CPU 运行时间为 1705ns。仿真波形和时序报告如下：



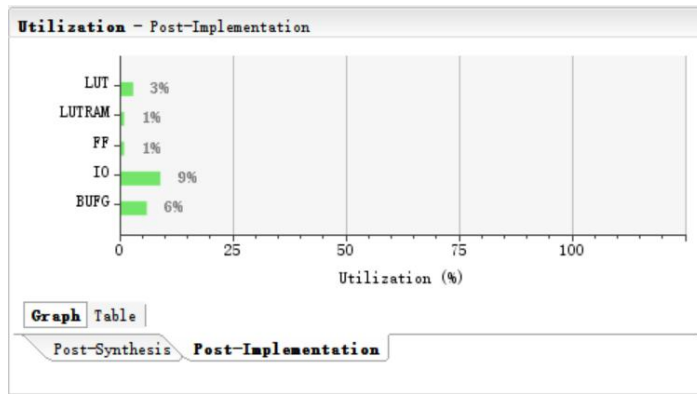
6、存储器空间的使用

寄存器 RegFile 有 32 个 32 位的寄存器， $32 \times 32\text{bits} = 128\text{B}$

IMEM 的 IP 核配置的是 $2048 \times 32\text{bits} = 8\text{KB}$

DMEM 同样是 32 个 32 位， $32 \times 32\text{bits} = 128\text{B}$

硬件资源使用率如下图：



九、总结与体会

本次设计一个五级流水线 CPU 的实验，是一次极具挑战性与收获的综合性的工程实践。从最初的指令集定义、模块化 Verilog 代码编写，到功能仿真、时序约束、综合实现，直至最终的 FPGA 下板调试，我完整地体验了现代处理器设计的核心流程。

在动手设计之前，我对流水线的理解停留在 IF-ID-EX-MEM-WB 五个阶段的抽象概念上。我曾简单地认为，流水线就是将多周期 CPU 的数据通路拉平，再在中间插入寄存器。但本次实践让我彻底颠覆了这一认知。流水线设计的本质，不是数据通路，而是控制通路；其核心不是执行，而是同步与协同。

五级流水线通过将指令执行并行化，极大地提高了 CPU 的吞吐率，但这是以急剧增加的复杂性为代价的。这种复杂性集中体现在流水线寄存器和冒险处理上。IF/ID、ID/EX、EX/MEM、MEM/WB 这四大流水线寄存器，如同工厂流水线上的“缓冲托盘”，它们不仅要传递数据，更要传递所有后续阶段所需的控制信号。

此外，本次设计采用了哈佛结构，将 IMEM 和 DMEM 分离。这在流水线中体现出了巨大的优越性：它从物理上解决了 IF 阶段（取指）和 MEM 阶段（访存）同时访问存储器的结构性冒险，使得取指和访存可以并行不悖，极大地保障了流水线的效率。

Hazard_Unit（冒险检测单元）是流水线 CPU 的“灵魂”。在设计中我深刻体会到，流水线 CPU 90% 的复杂度都集中在如何解决“冒险”上。

1. 数据冒险是流水线中最常遇到的问题。例如，ADD \$s0, \$s1, \$s2 之后紧跟 SUB \$s3, \$s0, \$s1。SUB 指令在 ID 阶段就需要 \$s0 的新值，但此时 ADD 指令才刚进入 EX 阶段，其结果要到 WB 阶段（两个周期后）才能写回 RegFile。

如果单纯依靠暂停，这将导致流水线性能大幅下降。因此，我设计了数据转发机制。Hazard_Unit 通过比较 ID 阶段的源寄存器（ID_rs, ID_rt）与 EX/MEM 和 MEM/WB 阶段的

目标寄存器（EX_MEM_write_reg, MEM_WB_write_reg），来判断是否需要转发。MEM -> EX 转发：当 Hazard_Unit 检测到 EX_MEM_reg_write 且寄存器匹配时，它会控制 EX 阶段的 ALU 输入 MUX，选择 EX_MEM_alu_result（来自上一条指令的 ALU 结果）作为输入，而不是从 ID_EX 寄存器中读取陈旧的数据。WB -> EX 转发同理，当检测到 MEM_WB_reg_write 匹配时，ALU 输入 MUX 会选择 WB_write_data（来自上上条指令的写回数据）。

2. Load-Use 冒险与“暂停” (Stall)。转发并非万能。LOAD 指令（如 LOAD \$s0, 0(\$s1)）在 MEM 阶段才能从 DMEM [cite: 462-465] 中取出数据。如果紧随其后的指令（如 ADD \$s2, \$s0, \$zero）在 EX 阶段就需要 \$s0，此时数据还在 DMEM 内部，即便是 MEM -> EX 转发也来不及。此时，Hazard_Unit 暂停取指和译码，并向 ID_EX 寄存器中注入一个“气泡”（即 NOP），使 LOAD 之后的 ADD 指令在 EX 阶段空转一个周期。当 ADD 指令再次进入 EX 阶段时，LOAD 指令已经位于 WB 阶段，此时就可以通过 WB -> EX 转发获取正确的数据了。

3. 控制冒险与“清空”。分支指令（BZ, BN）和跳转指令（JMP）带来了控制冒险。在本设计中，分支判断和目标地址计算都在 ID 阶段完成。但当 ID 阶段确认需要跳转时，IF 阶段已经“错误地”取了 PC+4 的指令。此时，Hazard_Unit 会检测 branch_taken 或 jump 信号，并立即触发 flush_IF_ID = 1 和 flush_ID_EX = 1。这两个清空信号会废弃掉流水线中处于 IF 和 ID 阶段的错误指令（将其替换为 NOP），同时 PCPU.v 中的 PC_next MUX 会选择正确的分支目标地址（branch_target）送入 PC，从而使流水线从正确的路径重新开始取指。

本次实验，通过从零开始设计、仿真、实现一颗五级流水线 CPU，我最大的收获不仅在于掌握了具体的 Verilog 代码实现技巧，更在于对计算机体系结构的核心思想、设计权衡与工程方法论有了更深刻的理解。

从设计一个自定义的指令集架构，到编写 Python 汇编器将汇编代码转换为机器码 coe 文件，再到设计 PCPU 来执行这些机器码，最后通过仿真验证其结果。这个端到端的工程闭环体验，让我对“软硬件协同”有了具象的理解。我不再是割裂地看待“软件”和“硬件”，而是明白了它们是如何联系在一起，协同完成一个计算任务。

通过本次实践，我不仅学会了流水线的实现细节，更重要的是，我学会了如何分析和解决流水线中的时空冲突，理解了性能与复杂度之间的权衡，并建立了一套严谨的工程验证方法。这份收获远超课程本身，是我在计算机学习道路上迈出的坚实一步。

十、附件（所有程序）

本项目的有关源文件均存放于根文件夹 CPU 设计源码下，并分为两大子文件夹：

1. 文件夹: CPU 源代码文件

此文件夹包含了构成流水线 CPU 硬件设计所需的所有 Verilog HDL 源文件。包括.v 文件、tb 文件、ip 核配置文件、xdc 文件。

注意！如果要用 tb 文件仿真，需要修改 PCPU 的时钟信号，并去掉七段数码管和分频器相关代码。

2. 文件夹: 验证文件 (汇编相关)

此文件夹包含了用于测试 CPU 功能、生成机器码和提供基准结果的所有验证相关文件。

注意！drop_eggs.asm 不能直接在 Mars 上运行，drop_eggs_mars.asm 是 drop_eggs.asm 的可以在 Mars 运行的等效版本。